

ComunidadESPOL

Plataforma web para la gestión y difusión de comunidades
estudiantiles de la ESPOL

INTEGRANTES

Silvia Milena Pazmiño Montes
Christian Gabriel Peláez Ávila
Carla Alejandra Gutiérrez Córdova

PROFESOR

Rodrigo Alexander Saraguro Bravo

MATERIA

Lenguajes de Programación

FACULTAD

Facultad de Ingeniería en Electricidad y Computación
(FIEC)

FECHA

28 de julio de 2026

LUGAR

Guayaquil, Ecuador

DESCRIPCIÓN DE LA PROPUESTA

Propuesta de ComunidadESPOL, una plataforma web para la gestión y difusión de las comunidades estudiantiles de la ESPOL. El documento detalla la problemática, los objetivos, el alcance, las características de la solución, los requerimientos funcionales por integrante, la arquitectura, la evaluación de lenguajes y el prototipo de baja fidelidad.

CONTENIDO

01	Problemática a resolver
02	Objetivos propuestos
03	Alcance del proyecto
04	Características de la solución
05	Requerimientos funcionales asignados
06	Arquitectura de la aplicación
07	Lenguajes de programación
08	Prototipo de baja fidelidad
09	Implementación
10	Resultados
11	Conclusiones
12	Recomendaciones
13	Referencias

1. Problemática a resolver

En la ESPOL existen decenas de comunidades estudiantiles (clubes técnicos, capítulos estudiantiles, grupos culturales y deportivos) cuya información se encuentra dispersa en cuentas de Instagram, grupos de WhatsApp y publicaciones aisladas en carteleras. Esto provoca que un estudiante de primeros niveles no tenga forma de conocer cuántas comunidades existen, a qué se dedican ni cómo integrarse a ellas, y que la difusión dependa exclusivamente de contactos personales. Por el lado de los líderes de comunidad, la administración de sus miembros se realiza en hojas de cálculo o listas informales que se pierden cada vez que cambia la directiva, sin un registro histórico de actividades ni de participación.

La ausencia de un espacio centralizado limita el crecimiento de las comunidades y dificulta que las unidades académicas dispongan de información confiable sobre la vida estudiantil extracurricular. Un sistema web institucional que centralice el catálogo de comunidades, gestione las solicitudes de afiliación y registre las actividades realizadas permitiría aumentar la visibilidad de los grupos, ordenar la

gestión interna de sus miembros y conservar la memoria de sus actividades entre generaciones de directivas.

2. Objetivos propuestos

Objetivo general:

Desarrollar una aplicación web que centralice la gestión y difusión de las comunidades estudiantiles de la ESPOL, permitiendo administrar su información, sus miembros y sus actividades.

Objetivos específicos:

- Implementar un módulo de registro y consulta de comunidades estudiantiles que permita publicar su perfil y explorarlas mediante filtros por categoría y facultad.
- Desarrollar un módulo de afiliación que gestione las solicitudes de ingreso de estudiantes a las comunidades y la consulta del padrón de miembros de cada una.
- Construir un módulo de actividades que permita registrar los eventos ejecutados por cada comunidad y consultar su historial de participación.

3. Alcance del proyecto

Se cubrirá:

- Aplicación web responsiva accesible desde navegador de escritorio y móvil.
- Dos roles: estudiante y líder de comunidad.
- Gestión del perfil de comunidades (crear, editar, listar, buscar y filtrar).
- Gestión de solicitudes de afiliación y del padrón de miembros.
- Registro de actividades por comunidad y consulta del historial de participación.
- Base de datos relacional poblada con datos de prueba (generados) representativos de comunidades de la ESPOL.

No se cubrirá:

- Integración con los sistemas institucionales de la ESPOL (SIDWEB, autenticación con correo institucional vía SSO). La autenticación implementada es local, con usuarios registrados en la base de datos del propio sistema.
- Aplicación móvil nativa.
- Control de asistencia a actividades y reportes estadísticos de participación.
- Carga de archivos e imágenes; los logos se referenciarán por URL.
- Procesamiento de pagos, cuotas o inscripciones con costo.
- Mensajería en tiempo real o chat interno entre miembros.
- Despliegue en infraestructura institucional; el sistema se desplegará en un servicio gratuito o entorno local para efectos de demostración.

4. Características de la solución

- **Catálogo público de comunidades:** vista principal con tarjetas de cada comunidad, buscador por nombre y filtros por categoría (técnica, cultural, deportiva, social) y facultad.
- **Perfil de comunidad:** descripción, logo, misión, redes de contacto, directiva actual y actividades próximas.
- **Panel del líder de comunidad:** creación y edición del perfil, revisión de solicitudes pendientes, administración del padrón y registro de actividades.
- **Flujo de afiliación:** el estudiante solicita ingreso desde el perfil de la comunidad y el líder aprueba o rechaza, con cambio de estado visible para ambas partes.
- **Módulo de actividades:** registro de eventos con fecha, lugar, tipo y responsable.
- **API REST:** expone todos los recursos y desacopla completamente el front-end del back-end.
- **Interfaz de una sola página (SPA):** navegación fluida y validaciones del lado del cliente.

5. Requerimientos funcionales asignados

Tabla 1. Requerimientos funcionales asignados por integrante.

Requerimiento	Descripción	Responsable
Registrar comunidad (escritura)	Permite al líder crear una comunidad con nombre, categoría, facultad, descripción, misión, logo y datos de contacto, y editarla posteriormente.	Milena
Explorar catálogo de comunidades (lectura)	Permite a cualquier usuario listar las comunidades activas y filtrarlas por categoría y facultad, además de buscar por nombre.	Milena
Solicitar y resolver afiliación (escritura)	Permite al estudiante enviar una solicitud de ingreso a una comunidad y al líder aprobarla o rechazarla.	Gabriel
Consultar padrón de miembros (lectura)	Permite al líder ver el listado de miembros de su comunidad con su rol, fecha de ingreso y estado, filtrable por estado de la solicitud.	Gabriel
Registrar actividad y asistencia (escritura)	Permite al líder registrar una actividad de su comunidad con título, tipo, fecha, lugar y descripción, y editarla o eliminarla.	Carla
Consultar historial y reporte de participación (lectura)	Permite a cualquier usuario ver el listado de actividades próximas y pasadas, filtrable por comunidad.	Carla

Cada integrante implementa tanto el endpoint en Laravel como la vista en React con TypeScript de sus dos requerimientos. El login, el diseño de interfaces y la base de datos se construyen de forma conjunta y no cuentan como requerimiento de nadie.

6. Arquitectura de la aplicación

Se ha definido la solución como una aplicación web porque es la más adecuada para cumplir con los objetivos del proyecto. Una aplicación web es accesible desde

cualquier navegador y dispositivo con conexión a internet. Esto es fundamental para llegar a toda la comunidad estudiantil de la ESPOL, quienes pueden acceder desde sus computadoras portátiles, tabletas o teléfonos inteligentes. Además, al ser una aplicación de tipo SPA (Single Page Application), se logra una experiencia de usuario fluida y rápida, similar a la de una aplicación de escritorio, pero con las ventajas de distribución y actualización centralizada de una web.

El patrón de arquitectura será API REST [5] + Cliente-Servidor con enfoque en el patrón MVC (Modelo-Vista-Controlador).

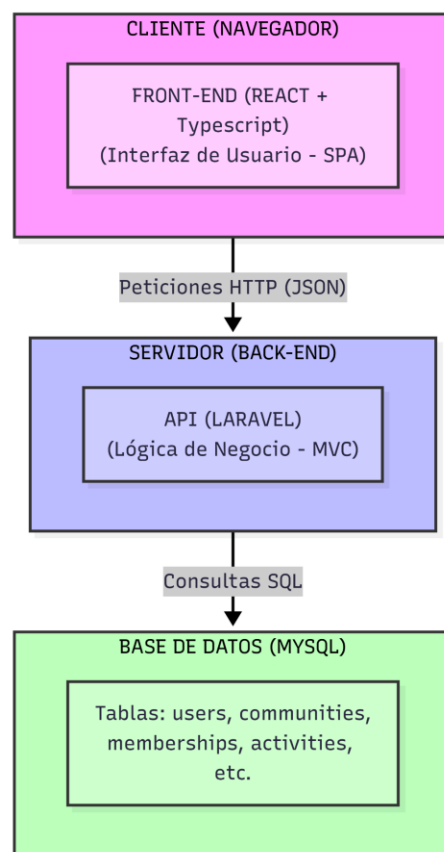


Figura 1. Patrón de arquitectura de la aplicación.

Herramientas y Frameworks:

- **Frameworks a usar:**

- Laravel es el framework principal para construir la API REST. Proporciona una estructura robusta para manejar la base de datos y herramientas integradas para autenticación, enrutamiento y validación, lo que acelera el desarrollo [2].
- React es la biblioteca elegida para construir la SPA. Permite crear interfaces de usuario modulares, reactivas y de alto rendimiento mediante el uso de componentes reutilizables y el Virtual DOM [4].
- **API:** Se implementará una API RESTful que expondrá todos los recursos del sistema a través de endpoints HTTP. Esta API será responsable de manejar las operaciones de lectura y escritura (crear, editar, listar, filtrar).
- **Servicios:**
 - Se implementó un sistema de autenticación local basado en sesiones de Laravel, que gestiona el inicio de sesión y la persistencia del estado del usuario para los dos roles del sistema mediante cookies seguras.
 - Se utilizará Eloquent, el ORM de Laravel, para realizar todas las consultas y operaciones sobre la base de datos de forma segura, evitando la construcción manual de sentencias SQL.
 - Se integrarán validaciones en el cliente (React con TypeScript) y en el servidor (Form Requests de Laravel) para garantizar que la información ingresada sea correcta y completa.

7. Lenguajes de programación

Los requerimientos de los lenguajes establecidos son: para el componente front-end se desarrollará en TypeScript, lenguaje previamente conocido por el grupo; y el componente back-end se desarrollará en PHP, lenguaje aprendido en la asignatura [1].

Back-end:

Primer lenguaje de programación: PHP con el framework Laravel.

- **Legibilidad:** media. Sintaxis familiar en estilo C, pero baja ortogonalidad y una biblioteca estándar históricamente inconsistente en nombres y orden de parámetros.
- **Escritura:** alta. Laravel aporta abstracciones (Eloquent, migraciones, validación declarativa) que permiten construir un CRUD completo con poco código.
- **Confiabilidad:** media-baja. Tipado dinámico y débil por defecto traslada a tiempo de ejecución errores que un lenguaje tipado detectaría antes. Se mitiga declarando tipos en las firmas desde PHP 8 [3] y usando Form Requests.
- **Costo:** bajo en entrenamiento, escritura y despliegue; ejecución interpretada con menor rendimiento que un compilado.

Segundo lenguaje de programación: Python con Django o Flask.

- **Legibilidad:** alta. Sintaxis limpia, indentación obligatoria y buena ortogonalidad.
- **Escritura:** alta, con excelente soporte para abstracción.
- **Confiabilidad:** media. Tipado dinámico pero fuerte, sin coerciones implícitas silenciosas.
- **Costo:** bajo en entrenamiento; no cumple el requisito de lenguaje aprendido en la asignatura.

Decisión: Se selecciona PHP con Laravel. Aunque Python presenta mejor legibilidad y ortogonalidad, PHP ofrece una escritura equivalente para el dominio del problema y un costo de entrenamiento menor gracias a Laravel, y es el lenguaje

que cumple el requisito de utilizar un LP aprendido en la asignatura. Su principal debilidad, la confiabilidad asociada al tipado dinámico, se mitigará declarando tipos en las firmas de métodos y centralizando la validación en Form Requests.

Front-end:

Primer lenguaje de programación: JavaScript.

- **Legibilidad:** media-baja. Coexistencia de paradigmas y sintaxis heredadas, (this) de comportamiento variable.
- **Escritura:** alta. Expresividad fuerte en eventos y asincronía (async/await).
- **Confiabilidad:** baja. Tipado dinámico y débil, con coerciones implícitas que producen errores difíciles de rastrear.
- **Costo:** el más bajo posible en el navegador, ya que no requiere traducción ni compilación.

Segundo lenguaje de programación: TypeScript [6].

- **Confiabilidad:** alta, por verificación de tipos en compilación.
- **Legibilidad:** alta, los tipos documentan los contratos entre componentes.
- **Costo:** bajo en el contexto del grupo. Transpilación y el esfuerzo de declarar tipos, compensado por la reducción de costo en depuración.

Decisión: se selecciona TypeScript. Dado que el grupo domina ambos lenguajes, el costo de entrenamiento, principal desventaja de TypeScript, se anula y el costo de configuración es marginal al utilizar plantillas de proyecto que integran el transpilador de forma predeterminada. En consecuencia, la evaluación se define por confiabilidad y legibilidad, criterios en los que TypeScript es superior: la verificación estática de tipos detecta en tiempo de compilación errores que JavaScript, por su tipado dinámico y débil, solo manifestaría en ejecución. Esta ventaja resulta particularmente relevante al desarrollar los requerimientos en

paralelo entre tres integrantes, ya que la declaración explícita de los tipos de las respuestas de la API actúa como contrato verificable entre el front-end y el back-end. Adicionalmente, TypeScript cumple el requisito de utilizar un LP ya conocido por el grupo.

8. Prototipo de baja fidelidad

- **Herramienta de Diseño Utilizada:** Figma [7]
- **Capturas:**

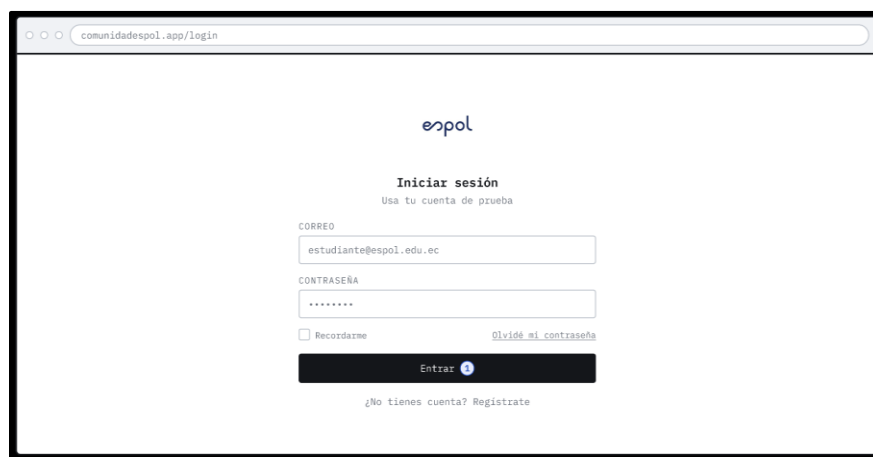


Figura 2. Prototipo de baja fidelidad: inicio de sesión.

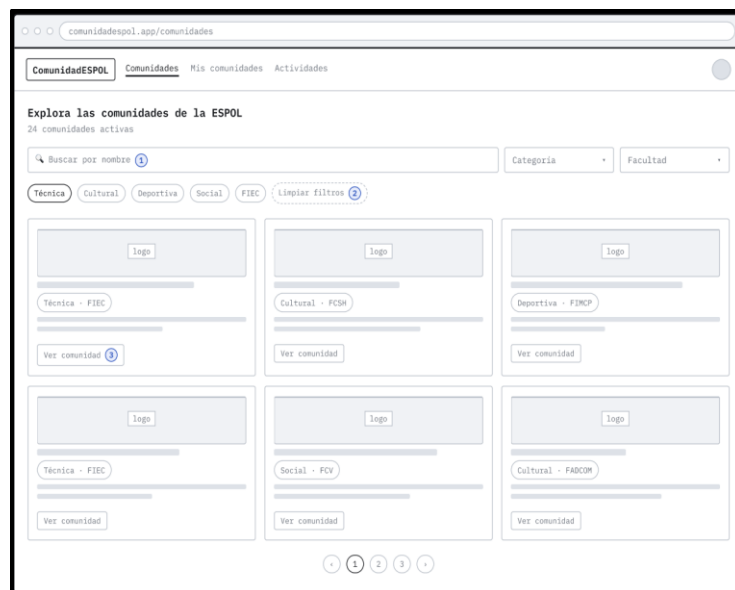


Figura 3. Prototipo de baja fidelidad: catálogo de comunidades.

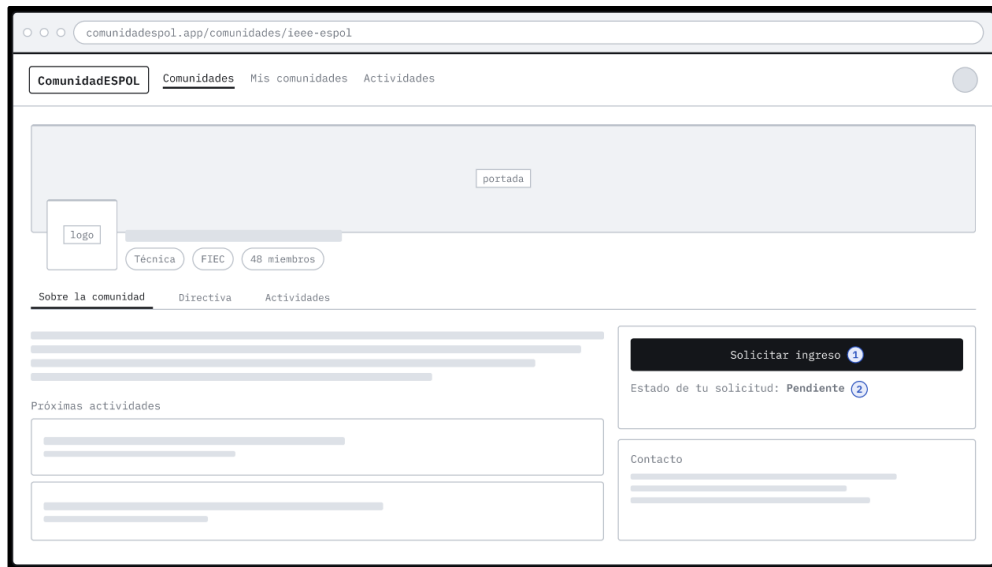


Figura 4. Prototipo de baja fidelidad: perfil de comunidad y solicitud de ingreso.

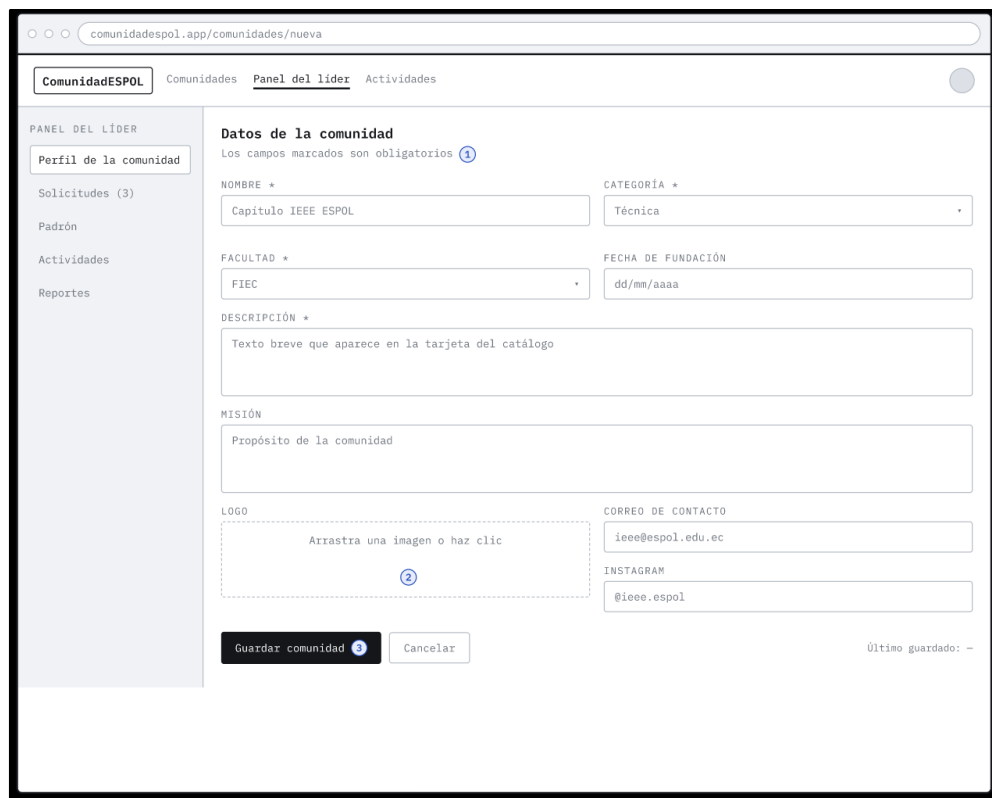


Figura 5. Prototipo de baja fidelidad: registrar o editar comunidad.

comunidadespol.app/panel/solicitudes

ComunidadESPOL Comunidades Panel del líder Actividades

PANEL DEL LÍDER

Perfil de la comunidad

Solicitudes (3)

Padrón

Actividades

Reportes

Solicitudes de ingreso

3 pendientes de revisión

Pendientes (3) Aprobadas (45) Rechazadas (2)

ESTUDIANTE	MATRÍCULA	FECHA	ESTADO	ACCIONES
		12/08/2026	Pendiente	<button>Aprobar</button> <button>Rechazar</button> ①
		12/08/2026	Pendiente	<button>Aprobar</button> <button>Rechazar</button>
		11/08/2026	Pendiente	<button>Aprobar</button> <button>Rechazar</button>

Al aprobar, el estudiante pasa al padrón como miembro activo ②

Figura 6. Prototipo de baja fidelidad: solicitudes de afiliación a comunidades.

comunidadespol.app/panel/actividades/nueva

ComunidadESPOL Comunidades Panel del líder Actividades

PANEL DEL LÍDER

Perfil de la comunidad

Solicitudes (3)

Padrón

Actividades

Reportes

Nueva actividad

Registra el evento y marca quién asistió

TÍTULO * TIPO *

FECHA * LUGAR *

DESCRIPCIÓN

RESPONSABLE

Asistencia ①

☒ Presidente

☒ Secretaria

☐ Miembro

☒ Miembro

3 de 48 miembros marcados

②

Figura 7. Prototipo de baja fidelidad: registrar actividad y asistencia.

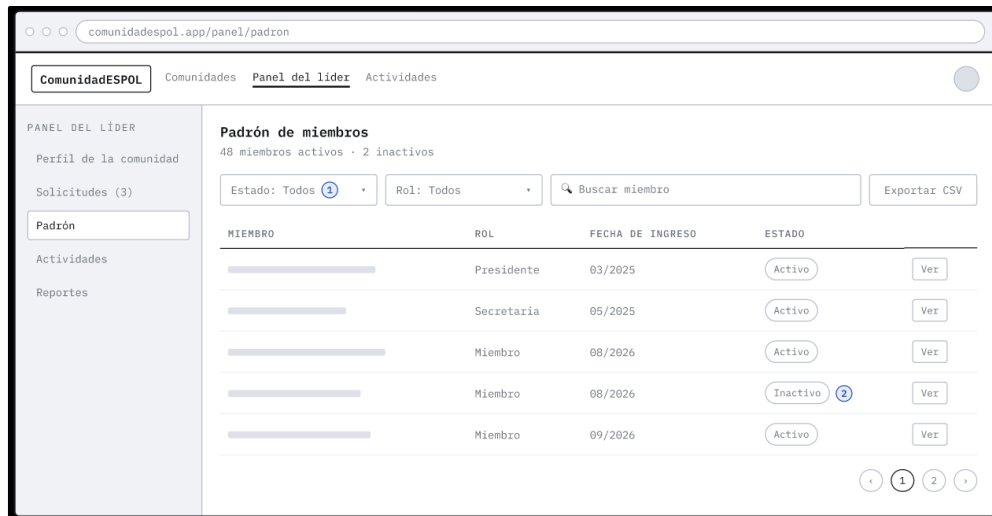


Figura 8. Prototipo de baja fidelidad: padrón de miembros.

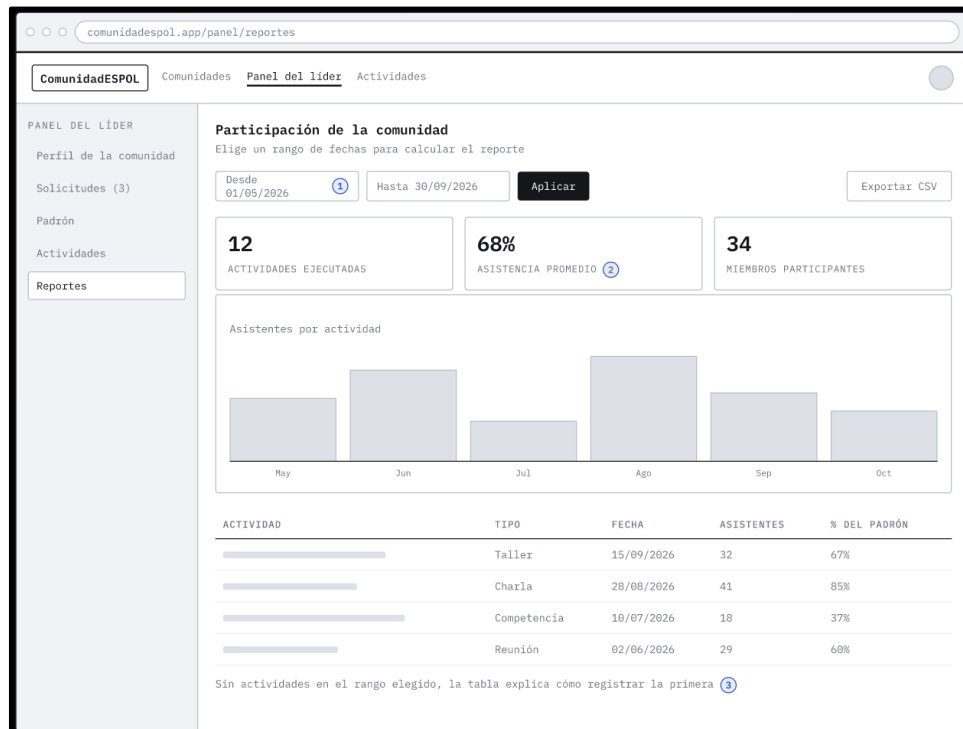


Figura 9. Prototipo de baja fidelidad: historial y reporte de participación.

- Enlace al Prototipo:

<https://www.figma.com/design/b5XP4TuTMQ8y0v33bJVeBd/Untitled?node-id=4-285&t=1FgUxhmTkC6sno83-1>

9. Implementación

Link de Repositorio: <https://github.com/gabriel12305/comunidad-espol.git>

9.1 Backend

El desarrollo del backend se realizó sobre un único proyecto Laravel compartido por los tres integrantes, con una sola base de datos MySQL. La estructura base: migraciones, modelos Eloquent y seeder de datos de prueba, se construyó de forma conjunta como punto de partida, ya que los tres módulos operan sobre las mismas tablas y relaciones. A partir de esa base, cada integrante implementó de forma independiente los controladores, las reglas de validación y las rutas correspondientes a sus dos requerimientos funcionales (uno de lectura y uno de escritura).

La API se expone bajo el prefijo /api y devuelve respuestas en formato JSON. Se emplean los verbos HTTP según su semántica REST: GET para lectura, POST para creación, PATCH para modificación parcial, PUT para actualización total y los códigos de estado correspondientes a cada resultado: 200 (OK), 201 (recurso creado), 403 (acceso denegado), 404 (recurso no encontrado), 409 (conflicto con el estado actual) y 422 (error de validación).

Estructura de datos implementada

Se definieron cuatro tablas mediante migraciones de Laravel: users, comunidades, membresías y actividades. Las relaciones entre ellas se declararon en los modelos Eloquent utilizando hasMany y belongsTo, lo que permite consultar datos asociados sin escribir sentencias SQL manuales. Adicionalmente, se implementó un seeder que puebla la base con tres usuarios, tres comunidades, sus membresías y actividades de prueba, garantizando que los tres integrantes trabajen sobre un conjunto de datos idéntico.

Tabla 2. Implementaciones del back-end y responsables.

Implementación	Responsable	Estado
Estructura base: migraciones, modelos Eloquent, relaciones y seeder de datos de prueba	Los tres integrantes	Hecho
Autenticación local: inicio y cierre de sesión sobre sesiones de Laravel y recuperación del usuario en sesión	Los tres integrantes	Hecho
Registrar y editar comunidad (escritura): POST /api/comunidades y PATCH /api/comunidades/{id}	Milena	Hecho
Explorar catálogo de comunidades (lectura): GET /api/comunidades con búsqueda por nombre y filtros por categoría y facultad	Milena	Hecho
Solicitar afiliación a una comunidad (escritura): POST /api/comunidades/{id}/solicitudes	Gabriel	Hecho
Resolver solicitud de afiliación (escritura): PATCH /api/comunidades/{id}/solicitudes/{id}	Gabriel	Hecho
Consultar padrón de miembros (lectura): GET /api/comunidades/{id}/miembros con filtros por estado y rol	Gabriel	Hecho
Registrar actividad y asistencia (escritura): POST /api/comunidades/{id}/actividades	Carla	Hecho
Consultar historial y reporte de participación (lectura): GET /api/comunidades/{id}/actividades	Carla	Hecho

Módulo de comunidades: Milena Pazmiño

Este módulo cubre el registro y la consulta de comunidades estudiantiles. Se implementaron cuatro endpoints en el controlador de comunidades, apoyados en Form Requests de Laravel para la validación de entrada.

El endpoint de listado construye la consulta de forma acumulativa sobre el constructor de consultas de Eloquent, aplicando únicamente los filtros presentes en la petición: categoría, facultad y búsqueda parcial por nombre. Cuando ningún

registro coincide con los criterios, devuelve una colección vacía con código 200 y no un 404, ya que la ausencia de resultados no constituye un error del cliente.

El endpoint de detalle recupera una comunidad por su identificador y devuelve 404 cuando el registro no existe. Los identificadores no numéricos son descartados antes de llegar al controlador, de modo que nunca se ejecuta una consulta con un valor inválido.

El endpoint de registro valida la presencia y el formato de los campos obligatorios y devuelve 422 con el detalle por campo cuando la validación falla. Verifica además la unicidad del nombre de la comunidad, reforzada por una restricción de unicidad en la base de datos, y la existencia del usuario indicado como líder, respondiendo 409 ante un nombre duplicado.

El endpoint de edición restringe la modificación al líder de la comunidad: compara el usuario de la petición con el propietario del registro y devuelve 403 cuando no coinciden. El mismo endpoint permite activar o desactivar la comunidad mediante el campo `activa`, de manera que las comunidades inactivas dejan de aparecer en el catálogo público sin eliminar el registro ni sus membresías asociadas.

Módulo de membresías: Gabriel Peláez

Este módulo gestiona el ciclo completo de afiliación de un estudiante a una comunidad. Se implementaron tres endpoints en `MembresiaController`, apoyados en la clase `StoreSolicitudRequest` para la validación de entrada.

El endpoint de solicitud verifica la existencia de la comunidad, impide solicitudes duplicadas mediante una consulta previa reforzada por una restricción de unicidad en la base de datos, y crea la membresía en estado pendiente. Los campos `estado` y `rol` se asignan de forma fija en el controlador y no se toman de la petición del usuario, como medida de protección frente a asignación masiva (`mass assignment`): de lo contrario, un estudiante podría enviar el campo `estado` con valor `aprobada` y auto-aprobar su ingreso sin intervención del líder.

El endpoint de resolución valida que el nuevo estado sea únicamente `aprobada` o `rechazada`, verifica que la solicitud continúe en estado pendiente, evitando así que

se altere una membresía ya consolidada. Devuelve 403 en caso contrario y al aprobar se registra automáticamente la fecha de ingreso.

El endpoint de consulta del padrón implementa filtros dinámicos opcionales por estado y por rol, contruidos de forma acumulativa sobre el constructor de consultas. Emplea carga anticipada (eager loading) de la relación con el usuario para evitar el problema de N+1 consultas, y restringe explícitamente las columnas devueltas para no exponer el hash de la contraseña en la respuesta JSON.

Módulo de actividades: Carla Gutiérrez

Este módulo gestiona el registro de las actividades ejecutadas por cada comunidad y la consulta de su historial. Se implementaron dos endpoints en el controlador de actividades, apoyados en Form Requests de Laravel para la validación de entrada.

El endpoint de registro crea una actividad asociada a la comunidad indicada en la ruta, con título, tipo, fecha, lugar y descripción. Valida la presencia y el formato de los campos obligatorios y devuelve 422 con el detalle por campo cuando la validación falla; el identificador de la comunidad se toma de la ruta y no del cuerpo de la petición, de modo que una actividad no pueda asignarse a una comunidad distinta de la solicitada.

La creación está restringida al líder de la comunidad: el controlador compara el usuario de la petición con el propietario del registro y devuelve 403 cuando no coinciden, evitando que un estudiante afiliado publique actividades en nombre de la directiva.

El endpoint de consulta devuelve el historial de actividades de una comunidad ordenado por fecha, diferenciando las próximas de las ya ejecutadas, y emplea carga anticipada (eager loading) de la relación con la comunidad para evitar el problema de N+1 consultas. Es de acceso libre, ya que el historial forma parte de la información pública del perfil de la comunidad.

9.2 Frontend

El componente de presentación se desarrolló como una aplicación de página única (SPA) construida con React y TypeScript sobre Vite, que consume la API REST

expuesta por el backend bajo el prefijo /api. Reside en el directorio frontend/ del mismo repositorio, pero constituye una aplicación autónoma: se compila por separado y se ejecuta en el navegador del usuario, comunicándose con el servidor exclusivamente mediante peticiones HTTP con respuestas en formato JSON. Esta separación materializa la arquitectura cliente-servidor descrita en la sección 6, en la que la capa de presentación queda por completo del lado del cliente.

Al igual que en el backend, se construyó primero una base compartida por los tres integrantes y, sobre ella, cada uno implementó de forma independiente las vistas correspondientes a sus dos requerimientos funcionales. Esa base comprende la configuración inicial del proyecto, el enrutamiento entre vistas, el layout general de navegación y cuatro piezas centrales:

Definición de tipos. El archivo `src/types/index.ts` declara las interfaces `User`, `Comunidad`, `Membresia` y `Actividad`, cuyos campos reproducen exactamente las columnas definidas en las migraciones, así como las estructuras de las respuestas de la API. Estas declaraciones operan como un contrato verificable entre el frontend y el backend: si una respuesta no coincide con el tipo esperado, el error se manifiesta en tiempo de compilación y no durante la ejecución. Los estados y roles de membresía se declararon como tipos de unión de literales (`'pendiente' | 'aprobada' | 'rechazada'`), lo que impide asignar valores distintos a los que acepta la validación del servidor.

Cliente HTTP. El archivo `src/api/client.ts` centraliza todas las peticiones a la API. Establece de forma uniforme las cabeceras `Accept` y `Content-Type`, necesarias para que Laravel responda en JSON incluso ante errores de validación; lee la URL base desde una variable de entorno y normaliza el tratamiento de errores mediante una clase `ApiError` que conserva el código de estado HTTP y los mensajes de validación devueltos por el servidor. De este modo, los códigos 403, 404, 409 y 422 se traducen en mensajes comprensibles para el usuario en lugar de fallos silenciosos. Esta centralización resuelve además una particularidad de la API `fetch` del navegador: los códigos de error no generan excepciones automáticamente, por lo que deben verificarse de forma explícita.

Estilos base. El archivo `src/styles/global.css` define las variables de color, tipografía y espaciado empleadas por los tres integrantes. Los estilos de cada pantalla se implementaron con CSS Modules, mecanismo que genera nombres de clase únicos en tiempo de compilación y garantiza que los estilos de un componente no interfieran con los de otro.

Sesión y protección de rutas. El archivo `src/context/AuthContext.tsx` conserva el usuario autenticado y su rol, obtenidos del endpoint de sesión de Laravel tras el inicio de sesión, y los expone al resto de la aplicación. Sobre ese contexto se construyó un componente de ruta protegida que redirige a la vista de inicio de sesión cuando no existe sesión activa y restringe las rutas del panel del líder según el rol, de modo que ninguna vista consulta la identidad del usuario desde un valor codificado.

En los formularios, las validaciones del lado del cliente replican las reglas definidas en los Form Requests de Laravel, de manera que el usuario reciba retroalimentación inmediata; no obstante, la validación del servidor se mantiene como la garantía efectiva, dado que la del cliente puede ser eludida.

Tabla 3. Implementaciones del front-end y responsables.

Implementación	Responsable	Estado
Base compartida: definición de tipos, cliente HTTP, estilos globales y configuración de rutas	Los tres integrantes	Hecho
Vista de inicio de sesión, manejo de la sesión del usuario y protección de rutas según rol	Los tres integrantes	Hecho
Vista de catálogo de comunidades (lectura): listado con buscador por nombre y filtros por categoría y facultad	Milena	Hecho
Formulario de registro y edición de comunidad (escritura), con validaciones del lado del cliente	Milena	Hecho
Perfil de comunidad con solicitud de ingreso y estado de la solicitud	Gabriel	Hecho

Implementación	Responsable	Estado
Bandeja de solicitudes de ingreso con aprobación y rechazo	Gabriel	Hecho
Padrón de miembros con filtros por estado y rol	Gabriel	Hecho
Formulario de registro de actividad y asistencia (escritura)	Carla	Hecho
Vista de historial y reporte de participación (lectura), diferenciando actividades próximas y pasadas	Carla	Hecho

10. Resultados

En esta sección cada integrante presenta la evidencia final de la funcionalidad que implementó en el componente de presentación (front-end), acompañada de la descripción de su comportamiento. Las evidencias parciales de los avances previos fueron retiradas y sustituidas por este apartado.

10.1 Módulo de comunidades: Silvia Milena Pazmiño Montes

Se implementaron dos pantallas que cubren el registro y la consulta de comunidades, apoyadas en una capa de servicios (`src/services/comunidades.ts`) que encapsula las llamadas a los endpoints del módulo, de modo que ningún componente construye directamente una URL de la API.

El catálogo de comunidades (`/comunidades`) lista las comunidades activas y ofrece un buscador por nombre y filtros combinables por categoría y facultad. Los criterios se traducen en parámetros de consulta que el endpoint interpreta de forma acumulativa, por lo que una sola vista cubre todas las combinaciones posibles sin requerir peticiones distintas. La ausencia de coincidencias se presenta como un mensaje explícito de resultado vacío y no como un error, en correspondencia con el código 200 que devuelve la API cuando ningún registro cumple los criterios.

El formulario de registro y edición de comunidad reutiliza un mismo componente para ambas operaciones: cuando recibe un identificador, precarga los datos

existentes y envía una actualización; en caso contrario, crea un nuevo registro. Las validaciones del lado del cliente replican las reglas definidas en los Form Requests, y los errores devueltos por el servidor se muestran campo por campo, de manera que un 422 se traduce en retroalimentación puntual sobre el campo afectado y un 409 por nombre duplicado en un mensaje sobre el campo nombre. El control del campo activa permite al líder retirar la comunidad del catálogo público sin eliminar el registro ni sus membresías asociadas.

En ambas pantallas se implementaron de forma explícita los estados de carga, de error y de resultado vacío, y los botones se deshabilitan mientras una operación está en curso para impedir envíos duplicados.

Autenticación: la identidad del usuario se obtiene de la sesión activa gestionada por Laravel, de modo que el formulario asocia la comunidad al líder autenticado sin valores codificados en el cliente. El acceso a la pantalla de registro y edición está restringido por rol mediante la protección de rutas del front-end.

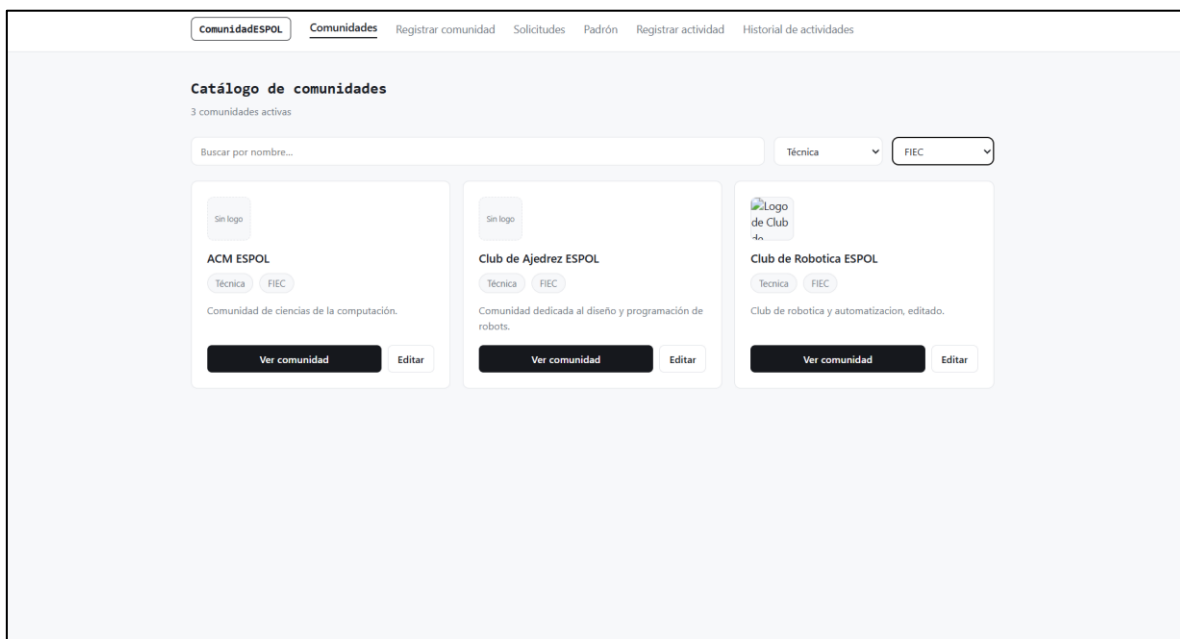


Figura 10. Catálogo de comunidades con filtros por categoría y facultad aplicados.

Funcionamiento: al seleccionar una categoría y una facultad en los desplegables, la vista compone los parámetros de consulta y ejecuta una nueva petición GET `/api/comunidades?categoria=...&facultad=...`. Mientras la respuesta está en tránsito se muestra el estado de carga; al recibirla, el listado se vuelve a renderizar

únicamente con las comunidades que satisfacen ambos criterios y el contador de resultados se actualiza. Los filtros son combinables entre sí y con el buscador por nombre, y al limpiarlos se restituye el catálogo completo.

10.2 Módulo de membresías: Christian Gabriel Peláez Ávila

Se implementaron tres pantallas que cubren el ciclo completo de afiliación, apoyadas en una capa de servicios (`src/services/membresias.ts`) que encapsula las llamadas a los tres endpoints del módulo. Ningún componente construye directamente una URL: esta separación permite modificar una ruta de la API en un solo punto del código.

El perfil de comunidad (`/comunidades/:id`) obtiene en paralelo los datos de la comunidad y el padrón, determina si el usuario ya posee una membresía y, en función de ello, habilita o deshabilita el botón de solicitud, mostrando el estado actual del trámite. Esta comprobación en el cliente evita solicitudes duplicadas que el servidor rechazaría con un código 409.

La bandeja de solicitudes (`/panel/solicitudes`) organiza las solicitudes en pestañas por estado y permite al líder aprobarlas o rechazarlas. Al resolverse una solicitud, la fila se retira de la vista y se muestra el mensaje de confirmación devuelto por el servidor, evitando duplicar en el cliente una lógica que ya reside en el backend.

El padrón de miembros (`/panel/padron`) presenta el listado completo con filtros combinables por estado y por rol. Los filtros se traducen en parámetros de consulta que el endpoint interpreta de forma acumulativa, de modo que una sola vista cubre todas las combinaciones posibles sin requerir peticiones distintas.

En las tres pantallas se implementaron de forma explícita los estados de carga, de error y de resultado vacío, y los botones se deshabilitan mientras una operación está en curso para impedir envíos duplicados.

Autenticación: la identidad del usuario se obtiene de la sesión activa gestionada por Laravel, por lo que las solicitudes de ingreso se registran a nombre del estudiante autenticado y la resolución de solicitudes queda reservada al líder de la comunidad. Las rutas del panel del líder están protegidas según el rol del usuario en sesión.

MIEMBRO	MATRÍCULA	ROL	FECHA DE INGRESO	ESTADO	ACCIONES
Sofía Vera	202201003	Miembro	—	Rechazada	
Ana Torres	202201001	Presidente	2025-03-01	Aprobada	
Luis Mora	202201002	Miembro	2025-08-15	Aprobada	

Figura 11. Padrón de miembros con el listado completo de la comunidad.

Funcionamiento: al ingresar a `/panel/padron` la vista solicita GET `/api/comunidades/{id}/miembros` y presenta cada membresía con el nombre del estudiante, su rol, la fecha de ingreso y el estado de la solicitud. Los desplega- bles de estado y de rol actúan como filtros combinables que se traducen en parámetros de consulta enviados al mismo endpoint, de modo que el filtrado se resuelve en el servidor y no en el cliente. Si la comunidad no tiene miembros que cumplan los criterios, se muestra el estado de resultado vacío en lugar de una tabla sin filas.

10.3 Módulo de actividades: Carla Alejandra Gutiérrez Córdova

Se implementaron dos vistas que cubren el registro de actividades y la consulta del historial de participación, apoyadas en un servicio centralizado (`actividades.ts`) que encapsula las llamadas a los endpoints de actividades de la API. La vista de historial (`/comunidades/:id/actividades`) presenta el listado con filtros combinables por tipo y período, sincronizados con la URL mediante query parameters. Los filtros se traducen en parámetros de consulta que el endpoint interpreta de forma acumulativa.

El formulario de registro (/panel/actividades/crear) replica las validaciones del servidor del lado del cliente, muestra errores campo por campo y deshabilita el botón durante el envío para evitar peticiones duplicadas.

Autenticación: la identidad del usuario se obtiene de la sesión activa gestionada por Laravel, de modo que el registro de una actividad se atribuye al líder autenticado y la validación del rol se resuelve en el servidor sobre el usuario de la sesión.

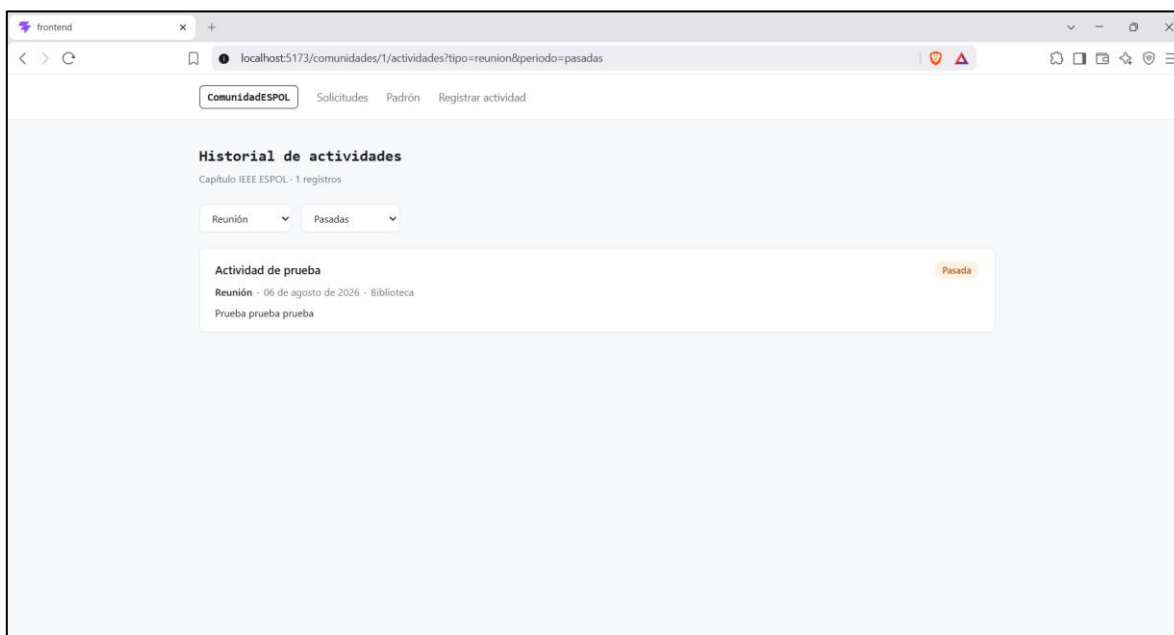


Figura 12. Historial de actividades con filtros aplicados y respuesta con coincidencias.

Funcionamiento: la vista de historial consulta GET `/api/comunidades/{id}/actividades` y separa las actividades en próximas y pasadas comparando su fecha con la fecha actual. Al aplicar un filtro se reenvía la petición con los parámetros seleccionados y la lista se reconstruye con las actividades coincidentes, ordenadas por fecha y con su título, tipo, lugar y responsable. Cuando la consulta no arroja coincidencias, la vista muestra un mensaje explícito de resultado vacío.

11. Conclusiones

11.1 Silvia Milena Pazmiño Montes

La aplicación de los criterios de evaluación de lenguajes de programación (legibilidad, facilidad de escritura, confiabilidad y costo) [1] permitió justificar técnicamente la elección de TypeScript sobre JavaScript en el front-end. Durante el desarrollo del catálogo y del formulario de comunidades, la verificación de tipos en tiempo de compilación detectó de forma temprana incompatibilidades entre los datos devueltos por la API y las interfaces declaradas en el cliente, errores que en JavaScript se habrían manifestado únicamente en tiempo de ejecución. Se concluye que el costo adicional de declarar tipos se compensa con la reducción del esfuerzo de depuración en un proyecto desarrollado por varias personas sobre una misma base de código.

11.2 Christian Gabriel Peláez Ávila

El uso de una arquitectura API REST [5] con separación estricta entre cliente y servidor resultó determinante para que los tres integrantes pudieran trabajar en paralelo: al fijar de antemano los contratos de los endpoints, el desarrollo del front-end y del back-end avanzó de manera independiente sin bloqueos mutuos. Asimismo, la duplicación deliberada de las validaciones en el cliente (React con TypeScript) y en el servidor (Form Requests de Laravel [2]) confirmó que la validación en el navegador mejora la experiencia de uso, pero no sustituye a la del servidor, que es la única capa que garantiza la integridad de los datos.

11.3 Carla Alejandra Gutiérrez Córdova

El desarrollo del módulo de actividades evidenció que las abstracciones que aporta el framework tienen un efecto directo sobre la facilidad de escritura del lenguaje: las migraciones, los modelos Eloquent y la validación declarativa de Laravel [2] permitieron implementar los endpoints con muy poco código, pese a que PHP [3] presenta por sí solo una legibilidad y una confiabilidad menores debido a su tipado dinámico y débil. Se concluye que la valoración de un lenguaje no puede desligarse de su ecosistema, y que la experiencia de construir el proyecto de

extremo a extremo (modelo, endpoint y vista en React [4]) fue lo que permitió comprender el costo real de cada decisión de diseño.

12. Recomendaciones

12.1 Silvia Milena Pazmiño Montes

Sobre la autenticación local ya implementada, se recomienda incorporar la recuperación de contraseña y el cierre de sesión por inactividad, y avanzar hacia la integración con el correo institucional mediante SSO [2], de modo que el estudiante no deba mantener credenciales propias de la plataforma. Se sugiere además reforzar la protección de rutas del front-end con una verificación de rol en cada petición al servidor, para que la restricción no dependa únicamente del cliente.

12.2 Christian Gabriel Peláez Ávila

Se recomienda incorporar paginación y ordenamiento en los endpoints de listado (catálogo de comunidades y padrón de miembros) y acompañarlos de pruebas automatizadas de la API, dado que el padrón crecerá con el número de afiliaciones y la carga completa del listado en una sola respuesta degradará el tiempo de respuesta a medida que aumenten los registros.

12.3 Carla Alejandra Gutiérrez Córdova

Se recomienda completar el control de asistencia por actividad y los reportes estadísticos de participación, que quedaron fuera del alcance de esta entrega, e integrarlos como indicadores visibles en el perfil de cada comunidad. Se sugiere también desplegar la aplicación en un entorno accesible en línea para validar su comportamiento con datos y usuarios reales.

13. Referencias

[1] R. W. Sebesta, Concepts of Programming Languages, 12th ed. Hoboken, NJ, USA: Pearson, 2019.

[2] Laravel, "Laravel Documentation," 2026. [En línea]. Disponible: <https://laravel.com/docs>

- [3] The PHP Group, "PHP Manual," 2026. [En línea]. Disponible:
<https://www.php.net/manual/es/>
- [4] Meta Open Source, "React Documentation," 2026. [En línea]. Disponible:
<https://react.dev>
- [5] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, Univ. of California, Irvine, CA, USA, 2000.
- [6] Microsoft, "TypeScript Documentation," 2026. [En línea]. Disponible:
<https://www.typescriptlang.org/docs/>
- [7] Figma Inc., "Figma Help Center," 2026. [En línea]. Disponible:
<https://help.figma.com/>